CNFSDP

Centre National de Formation en Statistique,
Démographie et Planification
Brazzaville, République du Congo

Introduction au Machine Learning

Guide du projet de groupe

Organisation, données DHS, fiches de démarrage et charte d'utilisation

Rois Céti DIMBAMBA

Ingénieur statisticien économiste · Actuaire · CV en ligne

Licence 3 · Semestre 5 · UE Informatique IV

Année académique 2026-2027

Supports, notebooks et TP : `https://github.com/rdimbamba/ml-cnfsdp`

Objet de ce guide

Ce guide accompagne le chapitre 5 (projet de bout en bout). Il contient tout ce qu'il faut pour mener le projet : l'organisation et le calendrier (section 1), la description des fichiers de l'enquête, de leur dictionnaire et des pondérations (section 2), la démarche attendue pour les volets supervisé et non supervisé (section 3), une fiche détaillée par groupe avec son code Python de préparation (section 4), puis les règles d'accès aux données et la charte à signer (section 5).
Deux notebooks accompagnent ce guide dans le dossier `notebooks/` du dépôt : `Validation_croisee.ipynb` (les hyperparamètres de chaque modèle du cours et leur choix par validation croisée) et `Projets_demarrage.ipynb` (le code de préparation des neuf groupes, à lancer sur les fichiers reçus).

1 Organisation

— **Groupes** : 36 étudiants, 9 groupes de 4 constitués par l'enseignant (tirage au sort, avec un équilibre entre étudiantes et étudiants) ; un thème différent par groupe, attribué par l'enseignant. Chaque groupe désigne un responsable, interlocuteur de l'enseignant.

— **Données** : Enquête démographique et de santé du Congo 2011-2012 (EDSC-II, programme DHS), fichiers originaux.

— **Travail demandé** : sur un même thème, une question de prédiction (chapitres 1 à 3) et une analyse non supervisée des mêmes données (ACP et typologie, chapitre 4), suivies d'une courte discussion des limites.

— **Livrables** : un rapport de 10 pages et un notebook ; une soutenance de 15 minutes où chacun des quatre membres présente une partie. Répartition indicative des 10 pages :

  — introduction, données et préparation (question, population, construction de la cible, valeurs manquantes, taux de contrôle) : environ 2 pages ;

  — prédiction (au moins trois modèles comparés, hyperparamètres choisis par validation croisée, métriques adaptées, interprétation) : environ 4,5 pages ;

  — analyse non supervisée (ACP, typologie, lien avec la cible) : environ 2,5 pages ;

  — conclusion et limites : environ 1 page.

— **Calendrier** : remise du rapport et du notebook le **dimanche 22 novembre 2026** ; soutenances le **samedi 28 novembre 2026**.

— **Évaluation du cours** : rapport et notebook 60 %, soutenance 20 %, note individuelle de participation 20 % (interventions pendant les cours et les TP). Il n'y a pas d'examen écrit.

gr.	thème	fichier	cible à prédire	population
1	richesse : indice national	`HR`	ménage dans les 40 % les plus pauvres (`hv270` $\in \{1, 2\}$)	tous les ménages
2	richesse : urbain et rural	`HR`	idem, un modèle par milieu (`hv025`)	ménages urbains, ruraux
3	vaccination complète	`KR`	8 vaccins reçus (`h2` à `h9` $\in \{1, 2, 3\}$)	enfants de 12 à 23 mois
4	abandon DTC1 → DTC3	`KR`	DTC1 reçu (`h3` $\in \{1, 2, 3\}$), DTC3 non (`h7` $\notin \{1, 2, 3\}$)	12-23 mois ayant eu le DTC1
5	rougeole	`KR`	vaccin reçu (`h9` $\in \{1, 2, 3\}$)	enfants de 12 à 23 mois
6	retard de croissance	`KR`	`hw70` < -200	enfants mesurés
7	insuffisance pondérale	`KR`	`hw71` < -200	enfants mesurés
8	contraception moderne	`IR`	`v313` $= 3$	femmes en union
9	besoins non satisfaits	`IR`	`v626a` $\in \{1, 2\}$	femmes en union

La composition des groupes est communiquée aux étudiants par courriel ; elle ne figure pas dans la version publique de ce guide.

2 Les données de l'enquête

2.1 Quatre fichiers, quatre niveaux d'observation

L'EDSC-II a interrogé 11 632 ménages tirés au sort dans tout le pays, toutes les femmes de 15 à 49 ans de ces ménages, et a recueilli l'historique de leurs naissances. Les données sont livrées en quatre fichiers « recode », un par niveau d'observation : une ligne du tableau est un ménage, une personne, une femme ou un enfant selon le fichier.

archive	une ligne =	identifiant	groupes
`CGHR61DT.zip`	un ménage	`hhid`	1, 2
`CGPR61DT.zip`	un membre d'un ménage	`hhid` + `hvidx` (numéro de ligne)	1, 2 (facultatif)
`CGIR61DT.zip`	une femme de 15 à 49 ans enquêtée	`caseid`	8, 9
`CGKR61DT.zip`	un enfant né dans les 5 ans avant l'enquête	`caseid` + `midx`	3 à 7

Les noms de variables suivent une convention commune à toutes les EDS du monde : `hv...` pour les variables du ménage, `v...` pour la femme, `b...` pour l'historique des naissances, `m...` pour la grossesse et l'accouchement, `h...` pour la santé de l'enfant (vaccins, diarrhée), `hw...` pour l'anthropométrie de l'enfant. Dans le fichier `KR`, chaque enfant porte aussi toutes les variables de sa mère (`v...`) : une mère de deux enfants apparaît sur deux lignes.

2.2 Les six fichiers de chaque archive

Chaque archive contient six fichiers de même nom, avec six extensions. Le préfixe se lit ainsi : `CG` (Congo), `KR` (type de fichier), `6` (phase 6 du programme DHS), `1` (version).

fichier	contenu et usage	ouvrir avec
`.DTA`	**Les données**, au format Stata : une ligne par unité, une colonne par variable. Il contient aussi les libellés (en anglais) des variables et de leurs modalités. C'est le seul fichier à lire pour l'analyse.	Python (`pandas.read_stata`), R (`haven::read_dta`), Stata (`use`), SPSS (Fichier, Ouvrir, Données, type Stata)
`.MAP`	**Le dictionnaire complet** (voir ci-dessous) : toutes les variables, dans l'ordre du questionnaire, avec leur libellé et la liste de leurs codes.	éditeur de texte (Bloc-notes, Notepad++, VS Code) ou Python
`.FRQ`	**Tableaux de fréquences** de chaque variable : effectif de chaque code, sans pondération. Sert à vérifier sa lecture : vos effectifs doivent être identiques.	éditeur de texte
`.FRW`	Les mêmes tableaux de fréquences, **pondérés** (le W de *weighted* ; l'en-tête du fichier le précise). Sert à vérifier un pourcentage pondéré.	éditeur de texte
`.DO`	Programme Stata qui attache les libellés aux variables et aux codes (`label variable`, `label define`). Utile seulement sous Stata ; lisible comme un texte.	Stata ou éditeur de texte
`.DCT`	Dictionnaire Stata au format « infile » : position et longueur de chaque variable dans la version texte brut des données. Inutile ici, le `.DTA` étant déjà prêt.	éditeur de texte

Excel n'ouvre pas les `.DTA`. Si un fichier texte affiche des caractères étranges, le rouvrir avec l'encodage « Western (Windows 1252) » ou « Latin-1 ».

2.3 Le dictionnaire des variables

Le fichier `.MAP` est la référence pour choisir et coder ses variables. Pour chaque variable, il donne son nom (en majuscules dans le `.MAP`, en minuscules dans le `.DTA` : `H9` devient `h9`), son libellé en anglais, sa position et sa longueur dans le fichier texte brut (sans intérêt pour vous), puis, pour une variable codée, la liste des codes et de leur signification. Quelques repères :

— les libellés sont en anglais : DTC s'écrit *DPT*, rougeole *MEASLES*, quintile de richesse *wealth index* ;

— une variable marquée « NA » n'a pas été collectée au Congo : elle est vide ;

— les codes de non-réponse sont listés avec les autres : il faut les repérer avant tout calcul (paragraphe 2.6) ;

— dans l'archive `IR` seulement, un fichier `.DOC` décrit les particularités de l'enquête congolaise (variables ajoutées ou non collectées).

Le dictionnaire se consulte aussi en Python, de deux façons : par les libellés enregistrés dans le `.DTA`, ou en cherchant dans le texte du `.MAP`.

```python
import pandas as pd
fichier = "donnees/CGKR61FL.DTA"                  # nom exact à lire dans votre dossier
with pd.io.stata.StataReader(fichier) as lecteur:
    libelles = lecteur.variable_labels()          # {nom : libellé}
    codes = lecteur.value_labels()                # {nom de la liste : {code : libellé}}
print(libelles["h9"])                             # libellé d'une variable
print({k: v for k, v in libelles.items() if "MEASLES" in v.upper()})   # recherche

import re
texte = open("donnees/CGKR61FL.MAP", encoding="latin-1").read()        # le .MAP
```

```
debut = re.search(r"^H9\s", texte, flags=re.MULTILINE).start()        # ligne de H9
print(texte[debut:debut + 600])                    # la variable et ses codes
```

2.4 Lire un fichier en Python

On ne lit que les colonnes utiles (le fichier `KR` en compte plus de 1 000) et l'on garde les codes numériques, que l'on interprète avec le dictionnaire :

```
df = pd.read_stata("donnees/CGKR61FL.DTA", columns=["v005", "b5", "h9"],
                   convert_categoricals=False)
```

La fonction `lire_dhs` du bloc commun (paragraphe 3.3) fait cela pour vous et trouve seule le fichier dans le dossier `donnees/`. Travaillez de préférence sur votre ordinateur (VS Code ou Jupyter). Avec Google Colab, téléversez les fichiers dans la session (ils disparaissent à sa fermeture), jamais dans un dossier partagé.

2.5 Les pondérations

Pourquoi des poids ? L'EDS ne tire pas tous les ménages avec la même probabilité. L'échantillon est stratifié par département et par milieu, et tiré en deux degrés : des grappes (zones de dénombrement), puis des ménages dans chaque grappe. Certaines strates sont volontairement sur-représentées pour avoir assez d'observations, et la non-réponse varie d'une zone à l'autre. Chaque unité enquêtée reçoit donc un **poids de sondage** : l'inverse de sa probabilité d'être enquêtée, corrigé de la non-réponse, puis normalisé pour que la somme des poids égale le nombre d'unités enquêtées. Un ménage de poids 2 « représente » deux fois plus de ménages de la population qu'un ménage de poids 1.

Où sont-ils ? Les poids sont stockés multipliés par 10^6 (six décimales implicites) : on divise toujours par 1 000 000.

fichier	poids à utiliser	remarque
`HR` (ménages)	`hv005` / 10^6	part des ménages
`HR`, part de la population	`hv005` / 10^6 × `hv012`	`hv012` : nombre de membres (de droit) du ménage
`PR` (membres)	`hv005` / 10^6	le poids du ménage
`IR` (femmes)	`v005` / 10^6	poids de la femme
`KR` (enfants)	`v005` / 10^6	poids de la mère, porté par l'enfant

La proportion pondérée. Dans une population de n unités de poids w_i, la proportion pondérée de la cible $y_i \in \{0,1\}$ est

$$\hat{p} = \frac{\sum_{i=1}^{n} w_i\, y_i}{\sum_{i=1}^{n} w_i}.$$

Exemple : trois enfants, $y = (1,0,1)$ et $w = (0{,}5\,;\,2\,;\,1{,}5)$: $\hat{p} = (0{,}5 + 1{,}5)/4 = 50\,\%$, alors que la proportion brute vaut $2/3 \approx 66{,}7\,\%$. En Python : `np.average(y, weights=w)`.

Quand pondérer ?

— **Toujours** pour un chiffre qui décrit la population congolaise : taux de contrôle, taux de la cible par classe ou par région, comparaison avec le rapport officiel de l'EDSC-II. Sans poids, un taux national est biaisé (les strates sur-représentées pèsent trop).

— **Pas obligatoire** pour apprendre et évaluer un modèle de prédiction : le modèle apprend le lien entre x et y, que la sur-représentation d'une strate ne fausse pas si la région et le milieu figurent parmi les variables explicatives. Il est possible de pondérer l'apprentissage (`fit(X, y, sample_weight=w)`) ; dites dans le rapport ce que vous avez fait.

Les **taux de contrôle** des fiches (section 4) sont des proportions pondérées, calculées avec le poids indiqué dans chaque fiche. Votre cible, une fois construite, doit les retrouver à quelques dixièmes de point près : c'est la preuve que la population et la cible sont bien construites.

2.6 Codes spéciaux et valeurs manquantes

Avant tout calcul, repérer dans le `.MAP` les codes qui ne sont pas de vraies valeurs, et les remplacer par une valeur manquante (`NaN`) :

— **ne sait pas** : souvent `8`, `98` ou `998` ; **manquant** : `9`, `99` ou `999` ;

— **anthropométrie** (`hw70`, `hw71`, `hw72`) : `9996` taille hors limites, `9997` âge hors limites, `9998` mesure incohérente, `9999` manquant ; seules les valeurs inférieures à 9996 sont des scores ;

— **non concerné** : une case vide (`NaN` dans Python) signale une question non posée, par exemple `m14` (visites prénatales), posée seulement pour la dernière naissance, ou `b11` (intervalle avec la naissance précédente), vide pour un premier-né ;

— **échelles** : certaines variables sont multipliées par 10 ou 100 (`hw70` : score $\times 100$; `v438` : taille en millimètres ; `v445` : IMC $\times 100$).

Un code de non-réponse laissé tel quel fausse tout : un âge « 98 » ou une taille « 9999 » tire les moyennes et les modèles.

3 La démarche attendue

3.1 Le volet supervisé

1. **Construire la population et la cible** avec le code de votre fiche, puis **retrouver le taux de contrôle**. Décrire la cible (taux pondéré, par milieu et par région).
2. **Séparer** apprentissage et test avant toute autre étape (`train_test_split`, 20 à 25 % pour le test, `stratify=y`, `random_state` fixé).
3. **Assembler un pipeline** : imputation des valeurs manquantes (`SimpleImputer`, médiane), standardisation pour la régression logistique et les k plus proches voisins, puis le modèle. Tout ce qui apprend sur les données (imputation, standardisation) se fait dans le pipeline, pour éviter les fuites (chapitre 5).
4. **Comparer au moins trois modèles** au classifieur de référence (`DummyClassifier`) : régression logistique, arbre, forêt aléatoire, gradient boosting, k plus proches voisins. Choisir les **hyperparamètres par validation croisée** stratifiée à 5 plis sur l'échantillon d'apprentissage seulement : le notebook `Validation_croisee.ipynb` donne, pour chaque modèle, les hyperparamètres à régler et le code.
5. **Choisir des métriques adaptées** : les classes sont déséquilibrées (de 11 à 75 % de positifs selon les groupes) ; on rapporte l'AUC, le rappel, la précision et le score F_1, pas seulement l'exactitude, et l'on discute le seuil de décision.
6. **Évaluer une seule fois sur le test** le modèle retenu, puis **l'interpréter** : importance des variables par permutation, sens des effets (coefficients, dépendance partielle).
7. **Discuter les limites** : données de 2011-2012, association et non causalité, variables écartées, qualité des déclarations.

Fichier `KR` : les enfants d'une même mère se ressemblent. Pour une évaluation plus honnête, on peut garder tous les enfants d'une mère dans le même pli (`GroupKFold` avec `groups=caseid`, présenté dans le notebook de validation croisée).

3.2 Le volet non supervisé, pas à pas

À retenir

L'idée. Le volet non supervisé décrit la structure des données *sans regarder la cible* : quelles variables vont ensemble (ACP), quels profils d'individus se dégagent (typologie). La cible n'intervient qu'à la fin, comme **variable illustrative** : on regarde comment elle se répartit sur les axes et dans les classes. Les variables qui construisent les axes et les classes sont les **variables actives**.

1. **Choisir les variables actives** : celles de votre fiche, toutes numériques (les variables à modalités ordonnées, comme l'instruction de 0 à 3, sont gardées comme des nombres ; les autres sont transformées en indicatrices 0/1). Garder les lignes complètes.
2. **ACP normée** (`StandardScaler` puis `PCA`) : on standardise pour que chaque variable compte autant, quelle que soit son unité. Tracer l'éboulis des parts d'inertie et garder les q premiers axes (avant le « coude », ou 70 à 80 % d'inertie cumulée).
3. **Nommer les axes** à partir des corrélations entre variables actives et axes : un axe où l'instruction, la richesse et l'exposition aux médias sont fortement corrélées dans le même sens est un axe de « niveau socio-économique ».
4. **Projeter les individus** sur le premier plan, colorés selon la cible : les individus positifs sont-ils concentrés d'un côté d'un axe ?
5. **Typologie** : classification ascendante hiérarchique (CAH, méthode de Ward) sur les coordonnées des q premiers axes ; choisir le nombre de classes k sur le dendrogramme (un grand saut de hauteur) ; puis **consolider** par k-means en partant des centres des classes de la CAH : chaque individu rejoint le centre le plus proche, ce qui améliore l'homogénéité des classes.
6. **Décrire et nommer les classes** : effectif, moyenne de chaque variable active par classe (comparée à la moyenne générale), puis taux pondéré de la cible par classe. Donner un nom à chaque classe (« mères jeunes, instruites, urbaines »…).

Dans le rapport, on montre : l'éboulis et le choix de q, le tableau des corrélations et le nom des axes, le premier plan coloré par la cible, le dendrogramme et le choix de k, le profil des classes avec le taux de la cible par classe. On termine par le lien avec le volet supervisé : les variables qui séparent le mieux les classes sont-elles aussi les plus importantes dans le meilleur modèle ?

Le code ci-dessous réalise ces six étapes ; il est le même pour tous les groupes et se lance après le code de préparation de la fiche, qui fournit le tableau `Z` des variables actives et le tableau `illus` (cible et poids).

```
# Volet non supervisé, commun à tous les groupes : à lancer après le code du groupe
# (il utilise Z, les variables actives, et illus : la cible et le poids).
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import linkage, fcluster, dendrogram

cible = illus.columns[0]                    # variable illustrative : la cible
```

```python
Zs = StandardScaler().fit_transform(Z)       # ACP normée : centrer, réduire

# 1. ACP : part d'inertie de chaque axe (éboulis), on garde les premiers axes
acp = PCA().fit(Zs)
print("Inertie expliquée (%) :", (100 * acp.explained_variance_ratio_).round(1))
q = 5                                        # nombre d'axes gardés, à choisir sur l'éboulis
coord = acp.transform(Zs)[:, :q]

# 2. Lire les axes : corrélation de chaque variable active avec les 3 premiers axes
cor = pd.DataFrame(acp.components_[:3].T * np.sqrt(acp.explained_variance_[:3]),
                   index=Z.columns, columns=["F1", "F2", "F3"])
display(cor.round(2).sort_values("F1"))

# 3. Premier plan factoriel, individus colorés selon la cible (illustrative)
plt.scatter(coord[:, 0], coord[:, 1], c=illus[cible], cmap="coolwarm", s=4, alpha=0.4)
plt.xlabel("F1"); plt.ylabel("F2"); plt.title(f"Premier plan, couleur : {cible}")
plt.show()

# 4. CAH de Ward sur les q premiers axes (sur 5 000 individus au plus, pour la mémoire)
rng = np.random.default_rng(0)
ech = rng.choice(len(coord), size=min(5000, len(coord)), replace=False)
arbre = linkage(coord[ech], method="ward")
dendrogram(arbre, truncate_mode="lastp", p=30); plt.title("Dendrogramme"); plt.show()
k = 4                                        # nombre de classes, lu sur le dendrogramme
classes_cah = fcluster(arbre, t=k, criterion="maxclust")

# 5. Consolidation : k-means partant des centres des classes de la CAH, sur tous
centres = np.vstack([coord[ech][classes_cah == c].mean(axis=0)
                     for c in range(1, k + 1)])
km = KMeans(n_clusters=k, init=centres, n_init=1).fit(coord)
classe = pd.Series(km.labels_ + 1, index=Z.index, name="classe")

# 6. Décrire les classes : effectif, moyennes des actives, taux pondéré de la cible
profil = Z.groupby(classe).mean().round(2)
profil.insert(0, "effectif", classe.value_counts().sort_index())
profil.insert(1, f"{cible} (%)", illus.groupby(classe).apply(
    lambda d: round(taux_pondere(d[cible], d["poids"]), 1)))
display(profil)
```

3.3 Le bloc commun

À exécuter une seule fois, en tête du notebook, avant le code de votre groupe. Il définit la lecture des fichiers, le traitement des codes spéciaux, la proportion pondérée et la construction des bases prêtes à l'emploi : `X` (variables explicatives, une indicatrice par modalité des variables catégorielles, la modalité « manquant » comprise), `y` (cible), `w` (poids) pour le supervisé ; `Z` (variables actives) et `illus` (cible et poids) pour le non supervisé. Les valeurs manquantes restantes des variables numériques de `X` sont imputées dans le pipeline.

```python
# Bloc commun à tous les groupes : à exécuter une fois, avant le code du groupe.
import os, re, glob
import numpy as np
import pandas as pd

DOSSIER = "donnees"   # dossier où les 4 archives .zip ont été décompressées

def chemin_dhs(fichier):
    """Chemin du fichier Stata du fichier 'HR', 'IR', 'KR' ou 'PR'."""
    motif = re.compile(rf"^CG{fichier}61\w*\.DTA$", re.IGNORECASE)
    trouves = [f for f in glob.glob(os.path.join(DOSSIER, "**", "*"), recursive=True)
               if motif.match(os.path.basename(f))]
    if not trouves:
        raise FileNotFoundError(f"Pas de CG{fichier}61...DTA dans « {DOSSIER} »")
```

```
    return trouves[0]

def lire_dhs(fichier, colonnes):
    """Lit les colonnes demandées, en gardant les codes numériques."""
    chemin = chemin_dhs(fichier)
    with pd.io.stata.StataReader(chemin) as lecteur:
        disponibles = set(lecteur.variable_labels())
    absentes = [c for c in colonnes if c not in disponibles]
    if absentes:
        print("Absentes du fichier, ignorées :", absentes)
    df = pd.read_stata(chemin, columns=[c for c in colonnes if c in disponibles],
                       convert_categoricals=False)
    print(f"{os.path.basename(chemin)} : {df.shape[0]} lignes, {df.shape[1]} colonnes")
    return df

def manquant(serie, codes):
    """Remplace par NaN les codes spéciaux (ne sait pas, manquant, hors limites...)."""
    return serie.where(~serie.isin(codes))

def indicatrice(serie, codes, manquants=(99,)):
    """1 si le code est dans la liste, 0 sinon, NaN si le code est manquant."""
    x = serie.isin(codes).astype(float)
    return x.where(serie.notna() & ~serie.isin(manquants))

def taux_pondere(y, poids):
    """Proportion pondérée, en % : somme(poids * y) / somme(poids)."""
    return 100 * np.average(y, weights=poids)

def base_supervisee(df, cible, numeriques, categorielles):
    """X (une indicatrice par modalité des catégorielles), cible y, poids w."""
    num = [c for c in numeriques if c in df]
    cat = [c for c in categorielles if c in df]
    X = df[num + cat].copy()
    for c in cat:   # une modalité « manquant » plutôt que de perdre la ligne
        X[c] = X[c].astype("Int64").astype(str).replace("<NA>", "manquant")
    X = pd.get_dummies(X, columns=cat, dtype=int)
    return X, df[cible].astype(int), df["poids"]

def base_non_supervisee(df, actives, illustratives):
    """Variables actives (sans valeur manquante) et variables illustratives."""
    act = [c for c in actives if c in df]
    garde = df[act].notna().all(axis=1)
    print(f"Non supervisé : {garde.sum()} lignes complètes sur {len(df)}")
    return df.loc[garde, act], df.loc[garde, illustratives]

def resume(X, y, w):
    print("X :", X.shape, "| cible :", round(100 * y.mean(), 1), "% (non pondéré),",
          round(taux_pondere(y, w), 1), "% (pondéré)")
    manq = X.isna().sum()
    print("Valeurs manquantes à imputer :", dict(manq[manq > 0]))
```

4 Fiches de démarrage

Chaque fiche donne la question à traiter, la population (qui garder, et comment), la construction pas à pas de la cible, des variables explicatives suggérées avec leur codage, les variables à écarter (fuites d'information : variables qui contiennent déjà la réponse ou qui servent à la construire), le taux de contrôle avec son poids, et le volet non supervisé. Les listes de variables sont des points de départ : on peut en ajouter après lecture du dictionnaire, en justifiant le choix.

Le code de préparation suit chaque fiche. Il se lance après le bloc commun (paragraphe 3.3), affiche le taux de contrôle, puis fournit les bases prêtes à l'emploi : `X`, `y`, `w` pour le supervisé, `Z` et `illus` pour le non supervisé (à passer ensuite au code du volet non supervisé). Les mêmes codes sont regroupés dans le notebook `Projets_demarrage.ipynb`.

Groupe 1 – Richesse : indice national

Question. Peut-on repérer les ménages pauvres à partir de caractéristiques simples à recueillir (taille et composition du ménage, chef de ménage, région), sans observer leurs biens ? C'est la question du ciblage rapide d'un programme social.
Population. Tous les ménages du fichier `HR` : 11 632 lignes, une par ménage. Poids : `hv005`/10^6.
Cible. `hv270` est le quintile de l'indice de richesse national (de 1 : les 20 % les plus pauvres, à 5 : les plus riches). On pose `pauvre` $= 1$ si `hv270` $\in \{1, 2\}$ (les deux quintiles les plus bas, soit les 40 % les plus pauvres), 0 sinon. Les quintiles sont définis sur la population : la part de ménages pauvres n'est donc pas exactement 40 %.
Explicatives suggérées.

- taille du ménage `hv009` ; enfants de moins de 5 ans `hv014` ; femmes de 15-49 ans `hv010` et hommes éligibles `hv011` ;
- sexe du chef `hv219` (1 homme, 2 femme) ; âge du chef `hv220` (codes 98, 99 : manquant) ;
- instruction du chef `instr_chef` : variable `hv106` du fichier `PR` pour la personne dont le lien avec le chef `hv101` vaut 1 (0 aucune, 1 primaire, 2 secondaire, 3 supérieur ; 8 : manquant) ;
- département `hv024`, milieu `hv025` (1 urbain, 2 rural).

À écarter. Tous les biens et toutes les caractéristiques du logement (`hv201` à `hv247`) : ils servent à calculer l'indice, donc la cible. Le score `hv271` et les quintiles `hv270`.
Taux de contrôle. 40,8 % des ménages (poids `hv005`/10^6) et 39,7 % de la population (poids `hv005`/10^6 × `hv012`).
Non supervisé : reconstruire soi-même un indice de richesse. Variables actives : les biens possédés (0/1 : électricité `hv206`, radio `hv207`, télévision `hv208`, réfrigérateur `hv209`, vélo `hv210`, moto `hv211`, voiture `hv212`, téléphone fixe `hv221`, portable `hv243a`, montre `hv243b`, terre agricole `hv244`, bétail `hv246`, compte bancaire `hv247`) et le logement (eau améliorée, toilettes améliorées, sol, murs et toit en matériau fini, cuisine à l'électricité ou au gaz, personnes par pièce). L'ACP normée donne un premier axe où presque toutes les variables sont corrélées dans le même sens : c'est un gradient de richesse, et la coordonnée sur cet axe est votre indice. Questions : quelle part d'inertie porte cet axe ? Quelle est sa corrélation avec l'indice officiel `hv271` (construit par une méthode voisine) ? Vos quintiles concordent-ils avec `hv270` (tableau croisé) ? Puis typologie des conditions de vie (CAH et k-means) et part pondérée de ménages pauvres par classe.

```python
# Groupe 1 - Richesse : indice national (fichier HR, un ménage par ligne)
biens = ["hv206", "hv207", "hv208", "hv209", "hv210", "hv211", "hv212", "hv221",
         "hv243a", "hv243b", "hv244", "hv246", "hv247"]
logement = ["hv201", "hv205", "hv213", "hv214", "hv215", "hv216", "hv226"]
menage = ["hhid", "hv005", "hv009", "hv010", "hv011", "hv012", "hv014",
          "hv024", "hv025", "hv219", "hv220", "hv270", "hv271"]
hr = lire_dhs("HR", menage + biens + logement)

# Poids et cible
hr["poids"] = hr["hv005"] / 1e6                    # poids de sondage du ménage
hr["pauvre"] = hr["hv270"].isin([1, 2]).astype(int)   # 2 quintiles les plus bas
print("Ménages pauvres :", round(taux_pondere(hr["pauvre"], hr["poids"]), 1), "%")
print("Population pauvre :",
      round(taux_pondere(hr["pauvre"], hr["poids"] * hr["hv012"]), 1), "%")

# Instruction du chef de ménage (fichier PR : hv101 = 1 pour le chef)
pr = lire_dhs("PR", ["hhid", "hv101", "hv106"])
chef = pr[pr["hv101"] == 1].drop_duplicates("hhid")[["hhid", "hv106"]]
hr = hr.merge(chef.rename(columns={"hv106": "instr_chef"}), on="hhid", how="left")
hr["instr_chef"] = manquant(hr["instr_chef"], [8, 9])

# Codes spéciaux : âge du chef 98-99 (ne sait pas, manquant)
```

```python
hr["hv220"] = manquant(hr["hv220"], [98, 99])

# Variables actives du non supervisé : 1 = possède / logement de qualité
biens = [b for b in biens if b in hr]
for b in biens:
    hr[b] = manquant(hr[b], [9])                    # 0 = non, 1 = oui, 9 = manquant
hr["eau_amelioree"] = indicatrice(hr["hv201"], [11, 12, 13, 14, 21, 31, 41, 51, 71])
hr["wc_ameliore"] = indicatrice(hr["hv205"], [11, 12, 13, 15, 21, 22, 41])
for v, nom in [("hv213", "sol_fini"), ("hv214", "mur_fini"), ("hv215", "toit_fini")]:
    hr[nom] = indicatrice(hr[v], range(31, 40))    # codes 31 à 39 : matériau fini
hr["cuisine_propre"] = indicatrice(hr["hv226"], [1, 2, 3, 4])  # électricité, gaz
hr["pers_par_piece"] = hr["hv009"] / manquant(hr["hv216"], [0, 99])

# Base du supervisé : caractéristiques démographiques (ni biens ni logement)
X, y, w = base_supervisee(hr, "pauvre",
    numeriques=["hv009", "hv010", "hv011", "hv014", "hv220"],
    categorielles=["hv024", "hv025", "hv219", "instr_chef"])
resume(X, y, w)

# Base du non supervisé : biens et logement ; hv270, hv271 et la cible en illustratif
actives = biens + ["eau_amelioree", "wc_ameliore", "sol_fini", "mur_fini",
                   "toit_fini", "cuisine_propre", "pers_par_piece"]
Z, illus = base_non_supervisee(hr, actives, ["pauvre", "hv270", "hv271", "poids"])
display(X.head(), Z.describe().T.round(2))
```

Groupe 2 – Richesse : indices urbain et rural

Question. Les signes de la pauvreté sont-ils les mêmes en ville et à la campagne ? Vaut-il mieux un modèle par milieu ou un modèle unique ?
Population. Les ménages du fichier `HR`, séparés selon `hv025` : 3 150 ménages urbains et 8 482 ruraux (effectifs non pondérés : le milieu rural est sur-représenté dans l'échantillon). Poids : $\texttt{hv005}/10^6$.
Cible. Comme le groupe 1 : `pauvre` $= 1$ si `hv270` $\in \{1, 2\}$ (quintiles nationaux).
Explicatives suggérées. Celles du groupe 1, sans le milieu dans les modèles par milieu. On compare trois approches : un modèle urbain, un modèle rural, un modèle unique avec `hv025` parmi les variables (le code prépare les trois bases).
À écarter. Comme le groupe 1 : biens, logement, `hv271`.
Taux de contrôle. 13,9 % de la population urbaine et 86,5 % de la population rurale appartiennent aux 40 % les plus pauvres (poids $\texttt{hv005}/10^6 \times \texttt{hv012}$). Le code affiche aussi la part des ménages. Les classes sont donc très déséquilibrées, en sens inverse dans les deux milieux : l'exactitude seule n'a aucun sens ici.
Non supervisé : un indice par milieu. Mêmes variables actives que le groupe 1, mais une ACP sur les ménages urbains et une autre sur les ruraux. Questions : quelles variables portent le premier axe dans chaque milieu (en ville : réfrigérateur, télévision, voiture ? à la campagne : sol, radio, vélo ?) ; corrélation de chaque premier axe avec `hv271` ; part des ménages qui changent de quintile quand on calcule les quintiles à l'intérieur du milieu. Typologie dans chaque milieu, part de ménages pauvres par classe. Pour lancer le code non supervisé sur un milieu : `Z, illus = bases["urbain"]["Z"], bases["urbain"]["illus"]`.

```python
# Groupe 2 - Richesse : indices urbain et rural (fichier HR, un ménage par ligne)
biens = ["hv206", "hv207", "hv208", "hv209", "hv210", "hv211", "hv212", "hv221",
         "hv243a", "hv243b", "hv244", "hv246", "hv247"]
logement = ["hv201", "hv205", "hv213", "hv214", "hv215", "hv216", "hv226"]
menage = ["hhid", "hv005", "hv009", "hv010", "hv011", "hv012", "hv014",
          "hv024", "hv025", "hv219", "hv220", "hv270", "hv271"]
hr = lire_dhs("HR", menage + biens + logement)

# Poids, cible et milieu (hv025 : 1 = urbain, 2 = rural)
hr["poids"] = hr["hv005"] / 1e6
hr["pauvre"] = hr["hv270"].isin([1, 2]).astype(int)
hr["milieu"] = hr["hv025"].map({1: "urbain", 2: "rural"})
for m, d in hr.groupby("milieu"):
    print(f"{m} : {len(d)} ménages ; pauvres",
```

```
          round(taux_pondere(d["pauvre"], d["poids"]), 1), "% des ménages,",
          round(taux_pondere(d["pauvre"], d["poids"] * d["hv012"]), 1),
          "% de la population")

# Instruction du chef de ménage (fichier PR : hv101 = 1 pour le chef)
pr = lire_dhs("PR", ["hhid", "hv101", "hv106"])
chef = pr[pr["hv101"] == 1].drop_duplicates("hhid")[["hhid", "hv106"]]
hr = hr.merge(chef.rename(columns={"hv106": "instr_chef"}), on="hhid", how="left")
hr["instr_chef"] = manquant(hr["instr_chef"], [8, 9])
hr["hv220"] = manquant(hr["hv220"], [98, 99])

# Variables actives (mêmes définitions que le groupe 1)
biens = [b for b in biens if b in hr]
for b in biens:
    hr[b] = manquant(hr[b], [9])
hr["eau_amelioree"] = indicatrice(hr["hv201"], [11, 12, 13, 14, 21, 31, 41, 51, 71])
hr["wc_ameliore"] = indicatrice(hr["hv205"], [11, 12, 13, 15, 21, 22, 41])
for v, nom in [("hv213", "sol_fini"), ("hv214", "mur_fini"), ("hv215", "toit_fini")]:
    hr[nom] = indicatrice(hr[v], range(31, 40))
hr["cuisine_propre"] = indicatrice(hr["hv226"], [1, 2, 3, 4])
hr["pers_par_piece"] = hr["hv009"] / manquant(hr["hv216"], [0, 99])
actives = biens + ["eau_amelioree", "wc_ameliore", "sol_fini", "mur_fini",
                   "toit_fini", "cuisine_propre", "pers_par_piece"]

# Une base par milieu : supervisé (sans hv025) et non supervisé
bases = {}
for m in ["urbain", "rural"]:
    d = hr[hr["milieu"] == m]
    X, y, w = base_supervisee(d, "pauvre",
        numeriques=["hv009", "hv010", "hv011", "hv014", "hv220"],
        categorielles=["hv024", "hv219", "instr_chef"])
    Z, illus = base_non_supervisee(d, actives, ["pauvre", "hv270", "hv271", "poids"])
    bases[m] = {"X": X, "y": y, "w": w, "Z": Z, "illus": illus}
    print(m, end=" : "); resume(X, y, w)
# Modèle unique, pour comparer : tous les ménages, avec le milieu hv025 parmi les X
X_tous, y_tous, w_tous = base_supervisee(hr, "pauvre",
    numeriques=["hv009", "hv010", "hv011", "hv014", "hv220"],
    categorielles=["hv024", "hv025", "hv219", "instr_chef"])
display(bases["urbain"]["X"].head(), bases["rural"]["Z"].describe().T.round(2))
```

Groupe 3 – Vaccination complète

Question. Quels enfants de 12 à 23 mois risquent de ne pas avoir reçu tous les vaccins de base ? Le but est de mieux cibler les relances.

Population. Le fichier `KR` contient tous les enfants nés dans les cinq ans avant l'enquête des femmes interrogées. On garde les enfants vivants (`b5` $= 1$) âgés de 12 à 23 mois. L'âge en mois se calcule par `v008` $-$ `b3` : les deux dates (entretien et naissance) sont en « mois depuis janvier 1900 ». Effectif : 1 884 enfants. Poids : `v005`$/10^6$ (celui de la mère).

Cible, pas à pas.

1. Les huit vaccins de base : BCG `h2` ; DTC 1, 2, 3 : `h3`, `h5`, `h7` ; polio 1, 2, 3 : `h4`, `h6`, `h8` ; rougeole `h9`. La polio à la naissance (`h0`) ne compte pas.
2. Codes de chaque vaccin : 0 non reçu ; 1 date notée sur la carte ; 2 déclaré par la mère ; 3 marqué sur la carte sans date ; 8 ne sait pas. Un vaccin est **reçu** si son code vaut 1, 2 ou 3 (carte ou déclaration, comme dans le rapport officiel) ; 8 compte comme non reçu.
3. `vacc_complete` $= 1$ si les huit vaccins sont reçus, 0 sinon.

Explicatives suggérées.

— enfant : âge en mois, sexe `b4`, rang de naissance `bord` ;

— mère : âge `v012`, instruction `v106` (0 à 3), religion `v130`, exposition aux journaux, à la radio, à la télévision `v157`, `v158`, `v159` (0 jamais à 3 presque chaque jour) ;

— soins : visites prénatales `m14` (posée seulement pour la dernière naissance, 98 : ne sait pas), accouchement en établissement de santé (`m15` de 20 à 49 : établissement public ou privé ; 11, 12 : domicile) ;

— ménage : quintile de richesse `v190`, taille `v136`, sexe du chef `v151`, milieu `v025`, département `v024`.

À écarter. Toutes les variables de vaccination `h0` à `h10` (elles composent la cible) et la possession d'une carte de vaccination `h1`, qui n'existe que parce que l'enfant a été vacciné.
Taux de contrôle. 38,1 % des enfants de 12-23 mois (poids `v005`/10^6). Le code affiche aussi la couverture de chaque vaccin : comparez-la au rapport officiel.
Non supervisé : profils de mères et de ménages. Variables actives : âge, instruction et exposition aux médias de la mère, quintile de richesse, milieu urbain, taille du ménage, rang de l'enfant, visites prénatales, accouchement en établissement. Le premier axe de l'ACP oppose probablement les mères instruites, aisées, urbaines, suivies pendant la grossesse aux autres : c'est un axe d'accès aux services. Questions : les enfants non complètement vaccinés se concentrent-ils d'un côté de cet axe ? Quelles classes de la typologie ont le taux de vaccination complète le plus bas, et comment les décrire ? Ce sont les profils à cibler.

```python
# Groupe 3 - Vaccination complète (fichier KR, un enfant né depuis 5 ans par ligne)
vaccins = ["h2", "h3", "h4", "h5", "h6", "h7", "h8", "h9"]
enfant = ["caseid", "midx", "v005", "v008", "b3", "b4", "b5", "bord"]
mere = ["v012", "v106", "v190", "v025", "v024", "v130", "v136", "v151",
        "v157", "v158", "v159", "m14", "m15"]
kr = lire_dhs("KR", enfant + mere + vaccins)

# Population : enfants vivants (b5 = 1) de 12 à 23 mois ; âge = v008 - b3 (en mois)
kr["age_mois"] = kr["v008"] - kr["b3"]
df = kr[(kr["b5"] == 1) & kr["age_mois"].between(12, 23)].copy()
df["poids"] = df["v005"] / 1e6           # poids de la mère, porté par l'enfant

# Cible : 8 vaccins reçus (1 = date sur carte, 2 = déclaré, 3 = marqué sur carte)
recu = df[vaccins].isin([1, 2, 3])
df["vacc_complete"] = recu.all(axis=1).astype(int)
print(len(df), "enfants de 12-23 mois ; vaccination complète :",
      round(taux_pondere(df["vacc_complete"], df["poids"]), 1), "%")
print("Couverture par vaccin (%) :",
      {v: round(taux_pondere(recu[v], df["poids"]), 1) for v in vaccins})

# Codes spéciaux et recodages
for v in ["v157", "v158", "v159"]:
    df[v] = manquant(df[v], [9])                  # 0 jamais ... 3 presque chaque jour
df["m14"] = manquant(df["m14"], [98, 99])  # visites prénatales (dernière naissance)
df["accouch_etab"] = indicatrice(df["m15"], range(20, 50))   # né en établissement
df["v130"] = manquant(df["v130"], [99])
df["urbain"] = (df["v025"] == 1).astype(int)

# Base du supervisé ; à écarter : h0 à h10, h1 (carte), et tout autre vaccin
X, y, w = base_supervisee(df, "vacc_complete",
    numeriques=["age_mois", "bord", "v012", "v136", "m14", "v157", "v158", "v159",
                "accouch_etab"],
    categorielles=["b4", "v106", "v190", "v025", "v024", "v130", "v151"])
resume(X, y, w)

# Base du non supervisé : profil de la mère et du ménage ; la cible en illustratif
actives = ["v012", "v106", "v190", "urbain", "v136", "bord", "v157", "v158", "v159",
           "m14", "accouch_etab"]
Z, illus = base_non_supervisee(df, actives, ["vacc_complete", "poids"])
display(X.head(), Z.describe().T.round(2))
```

Groupe 4 – Abandon entre DTC1 et DTC3

Question. Parmi les enfants qui ont commencé le calendrier vaccinal, lesquels ne l'achèvent pas ? L'abandon mesure la capacité du système de santé à faire revenir les familles.
Population. Enfants vivants de 12 à 23 mois du fichier `KR` (comme le groupe 3) **ayant reçu le DTC1** (`h3` $\in \{1, 2, 3\}$) : 1 290 enfants. Poids : `v005`$/10^6$.
Cible, pas à pas. DTC1 reçu si `h3` $\in \{1, 2, 3\}$; DTC3 reçu si `h7` $\in \{1, 2, 3\}$ (mêmes codes que le groupe 3). `abandon` $= 1$ si le DTC3 n'a pas été reçu, 0 sinon, parmi les enfants qui ont reçu le DTC1.
Explicatives suggérées. Celles du groupe 3. L'âge en mois compte : un enfant de 12 mois a pu recevoir son DTC1 en retard et n'être pas encore au bout du calendrier.
À écarter. Toutes les variables `h0` à `h10` et `h1` : les autres doses (polio 3, rougeole) se donnent souvent lors des mêmes visites que le DTC3 et annonceraient la réponse.
Taux de contrôle. 25,0 % d'abandon (poids `v005`$/10^6$) ; parmi tous les 12-23 mois, DTC1 : 74,5 %, DTC3 : 56,3 %.
Non supervisé : profils des familles qui ont commencé. Mêmes variables actives que le groupe 3, sur les seuls enfants ayant reçu le DTC1. Questions : quelles classes abandonnent le plus ? L'abandon suit-il le même axe socio-économique que l'absence de vaccination (groupe 3), ou d'autres facteurs (taille du ménage, rang de l'enfant, âge de la mère) ?

```python
# Groupe 4 - Abandon entre DTC1 et DTC3 (fichier KR, un enfant par ligne)
vaccins = ["h2", "h3", "h4", "h5", "h6", "h7", "h8", "h9"]
enfant = ["caseid", "midx", "v005", "v008", "b3", "b4", "b5", "bord"]
mere = ["v012", "v106", "v190", "v025", "v024", "v130", "v136", "v151",
        "v157", "v158", "v159", "m14", "m15"]
kr = lire_dhs("KR", enfant + mere + vaccins)

# Enfants vivants de 12 à 23 mois
kr["age_mois"] = kr["v008"] - kr["b3"]
base = kr[(kr["b5"] == 1) & kr["age_mois"].between(12, 23)].copy()
base["poids"] = base["v005"] / 1e6
base["dtc1"] = base["h3"].isin([1, 2, 3]).astype(int)
base["dtc3"] = base["h7"].isin([1, 2, 3]).astype(int)
print("12-23 mois :", len(base),
      "; DTC1 :", round(taux_pondere(base["dtc1"], base["poids"]), 1),
      "% ; DTC3 :", round(taux_pondere(base["dtc3"], base["poids"]), 1), "%")

# Population : ceux qui ont reçu le DTC1 ; cible : DTC3 non reçu (abandon)
df = base[base["dtc1"] == 1].copy()
df["abandon"] = 1 - df["dtc3"]
print(len(df), "enfants ayant reçu le DTC1 ; abandon :",
      round(taux_pondere(df["abandon"], df["poids"]), 1), "%")

# Codes spéciaux et recodages
for v in ["v157", "v158", "v159"]:
    df[v] = manquant(df[v], [9])
df["m14"] = manquant(df["m14"], [98, 99])
df["accouch_etab"] = indicatrice(df["m15"], range(20, 50))
df["v130"] = manquant(df["v130"], [99])
df["urbain"] = (df["v025"] == 1).astype(int)

# Base du supervisé ; à écarter : h0 à h10 (dont h3, h5, h7), h1 (carte)
X, y, w = base_supervisee(df, "abandon",
    numeriques=["age_mois", "bord", "v012", "v136", "m14", "v157", "v158", "v159",
                "accouch_etab"],
    categorielles=["b4", "v106", "v190", "v025", "v024", "v130", "v151"])
resume(X, y, w)

# Base du non supervisé ; l'abandon en illustratif
actives = ["v012", "v106", "v190", "urbain", "v136", "bord", "v157", "v158", "v159",
           "m14", "accouch_etab"]
Z, illus = base_non_supervisee(df, actives, ["abandon", "poids"])
display(X.head(), Z.describe().T.round(2))
```

Groupe 5 – Vaccination contre la rougeole

Question. Quels enfants de 12 à 23 mois n'ont pas reçu le vaccin contre la rougeole, maladie qui provoque encore des épidémies ?
Population. Enfants vivants de 12 à 23 mois du fichier `KR` : 1 884 enfants. Poids : `v005`/10^6.
Cible. `rougeole` = 1 si `h9` $\in \{1, 2, 3\}$ (date sur la carte, déclaration de la mère, marque sur la carte) ; 0 si `h9` vaut 0 (non) ou 8 (ne sait pas).
Explicatives suggérées. Celles du groupe 3. L'âge en mois compte particulièrement : le vaccin se donne à 9 mois, les enfants de 12 mois ont pu le recevoir en retard.
À écarter. Les autres vaccins (`h0` à `h8`, `h10`) et la carte `h1` : un enfant qui a reçu le DTC3 a presque toujours reçu la rougeole, ce serait une fuite.
Taux de contrôle. 74,9 % (poids `v005`/10^6).
Non supervisé : où sont les non-vaccinés ? Mêmes variables actives que le groupe 3. On insiste ici sur l'ACP : projeter les enfants sur le premier plan factoriel, colorés selon la cible ; lire les axes grâce aux corrélations ; dire si les non-vaccinés occupent une zone particulière du plan. Puis typologie et taux de vaccination par classe.

```python
# Groupe 5 - Vaccination contre la rougeole (fichier KR, un enfant par ligne)
enfant = ["caseid", "midx", "v005", "v008", "b3", "b4", "b5", "bord", "h9"]
mere = ["v012", "v106", "v190", "v025", "v024", "v130", "v136", "v151",
        "v157", "v158", "v159", "m14", "m15"]
kr = lire_dhs("KR", enfant + mere)

# Population : enfants vivants de 12 à 23 mois
kr["age_mois"] = kr["v008"] - kr["b3"]
df = kr[(kr["b5"] == 1) & kr["age_mois"].between(12, 23)].copy()
df["poids"] = df["v005"] / 1e6

# Cible : rougeole reçue (h9 = 1, 2 ou 3) ; 0 = non, 8 = ne sait pas comptent comme non
df["rougeole"] = df["h9"].isin([1, 2, 3]).astype(int)
print(len(df), "enfants de 12-23 mois ; rougeole :",
      round(taux_pondere(df["rougeole"], df["poids"]), 1), "%")

# Codes spéciaux et recodages
for v in ["v157", "v158", "v159"]:
    df[v] = manquant(df[v], [9])
df["m14"] = manquant(df["m14"], [98, 99])
df["accouch_etab"] = indicatrice(df["m15"], range(20, 50))
df["v130"] = manquant(df["v130"], [99])
df["urbain"] = (df["v025"] == 1).astype(int)

# Base du supervisé ; l'âge compte (vaccin vers 9 mois) ; à écarter : h0 à h10
X, y, w = base_supervisee(df, "rougeole",
    numeriques=["age_mois", "bord", "v012", "v136", "m14", "v157", "v158", "v159",
                "accouch_etab"],
    categorielles=["b4", "v106", "v190", "v025", "v024", "v130", "v151"])
resume(X, y, w)

# Base du non supervisé (ACP) ; la cible en illustratif
actives = ["v012", "v106", "v190", "urbain", "v136", "bord", "v157", "v158", "v159",
           "m14", "accouch_etab"]
Z, illus = base_non_supervisee(df, actives, ["rougeole", "poids"])
display(X.head(), Z.describe().T.round(2))
```

Groupe 6 – Retard de croissance

Question. Quels enfants de moins de 5 ans souffrent d'un retard de croissance, signe d'une malnutrition chronique ?
Population. L'anthropométrie n'a été mesurée que dans la moitié des ménages. On garde les enfants vivants (`b5` = 1) du fichier `KR` qui ont un score valide : `hw70` < 9996 (les codes 9996 à 9999 signalent une mesure hors limites, incohérente ou absente). Effectif : 4 475 enfants. Poids : `v005`/10^6.

Cible. `hw70` est le score z taille-pour-âge multiplié par 100 (normes de croissance de l'OMS) : -250 signifie que l'enfant est 2,5 écarts types sous la taille médiane des enfants de son âge et de son sexe. `retard` $= 1$ si `hw70` < -200, 0 sinon.
Explicatives suggérées.

- enfant : âge en mois `hw1` (le retard augmente avec l'âge), sexe `b4`, rang `bord`, intervalle avec la naissance précédente `b11` (vide pour un premier-né, d'où l'indicatrice `premier_ne`), jumeau (`b0` > 0), taille à la naissance perçue par la mère `m18` (1 très gros à 5 très petit ; 8 : ne sait pas), diarrhée récente (`h11` $\in \{1, 2\}$) ;
- mère : âge `v012`, instruction `v106`, taille `v438` (en millimètres, divisée par 10 ; codes ≥ 9990 : manquant), IMC `v445` (divisé par 100 ; codes ≥ 9990 : manquant) ;
- ménage : quintile `v190`, taille `v136`, eau améliorée (`v113`), toilettes améliorées (`v116`), milieu `v025`, département `v024`.

À écarter. Les autres indices anthropométriques `hw71`, `hw72` et les mesures brutes de l'enfant (poids `hw2`, taille `hw3`) : ils servent à calculer la cible ou en sont très proches.
Taux de contrôle. 23,1 % (poids `v005`$/10^6$, fichier `KR`). Le rapport officiel calcule ce taux sur les enfants présents la nuit précédente du fichier des membres (`PR`, poids `hv005`) : un écart d'un ou deux points avec lui est normal.
Non supervisé : profils enfant–mère–ménage. Variables actives : âge de l'enfant, rang, taille à la naissance, taille et IMC de la mère, instruction, quintile, milieu, taille du ménage, eau et toilettes. Questions : quels axes se dégagent (niveau de vie, corpulence de la mère, âge de l'enfant) ? Où se placent les enfants en retard de croissance sur le premier plan ? Quelles classes ont le taux de retard le plus élevé ?

```python
# Groupe 6 - Retard de croissance (fichier KR, un enfant par ligne)
enfant = ["caseid", "midx", "v005", "b0", "b4", "b5", "bord", "b11", "m18",
          "h11", "hw1", "hw70"]
mere = ["v012", "v106", "v190", "v025", "v024", "v136", "v113", "v116",
        "v438", "v445"]
kr = lire_dhs("KR", enfant + mere)

# Population : enfants vivants mesurés ; hw70 >= 9996 = hors limites ou manquant
df = kr[(kr["b5"] == 1) & (kr["hw70"] < 9996)].copy()
df["poids"] = df["v005"] / 1e6

# Cible : score taille-pour-âge (x 100) inférieur à -2 écarts types
df["retard"] = (df["hw70"] < -200).astype(int)
print(len(df), "enfants mesurés ; retard de croissance :",
      round(taux_pondere(df["retard"], df["poids"]), 1), "%")

# Codes spéciaux et recodages
df["taille_mere"] = manquant(df["v438"], range(9990, 10000)) / 10    # en cm
df["imc_mere"] = manquant(df["v445"], range(9990, 10000)) / 100      # en kg/m2
df["m18"] = manquant(df["m18"], [8, 9])   # taille à la naissance : 1 très gros ... 5
df["premier_ne"] = df["b11"].isna().astype(int)  # pas d'intervalle pour un premier-né
df["jumeau"] = (df["b0"] > 0).astype(int)
df["diarrhee"] = indicatrice(df["h11"], [1, 2], manquants=(8, 9))
df["eau_amelioree"] = indicatrice(df["v113"], [11, 12, 13, 14, 21, 31, 41, 51, 71])
df["wc_ameliore"] = indicatrice(df["v116"], [11, 12, 13, 15, 21, 22, 41])
df["urbain"] = (df["v025"] == 1).astype(int)

# Base du supervisé ; à écarter : hw70, hw71, hw72 (indices), hw2, hw3 (mesures)
X, y, w = base_supervisee(df, "retard",
    numeriques=["hw1", "bord", "b11", "m18", "v012", "v136", "taille_mere", "imc_mere",
                "premier_ne", "jumeau", "diarrhee", "eau_amelioree", "wc_ameliore"],
    categorielles=["b4", "v106", "v190", "v025", "v024"])
resume(X, y, w)

# Base du non supervisé : enfant, mère, ménage ; la cible en illustratif
actives = ["hw1", "bord", "m18", "taille_mere", "imc_mere", "v106", "v190", "urbain",
```

```
            "v136", "eau_amelioree", "wc_ameliore"]
Z, illus = base_non_supervisee(df, actives, ["retard", "poids"])
display(X.head(), Z.describe().T.round(2))
```

Groupe 7 – Insuffisance pondérale

Question. Quels enfants de moins de 5 ans ont un poids trop faible pour leur âge ?
Population. Enfants vivants du fichier `KR` avec un score valide `hw71` < 9996 : 4 475 enfants. Poids : `v005`$/10^6$.
Cible. `hw71` est le score z poids-pour-âge multiplié par 100. `insuff_pond` $= 1$ si `hw71` < -200, 0 sinon.
Explicatives suggérées. Celles du groupe 6 (même code, même codage).
À écarter. `hw70`, `hw72`, poids `hw2` et taille `hw3` de l'enfant.
Taux de contrôle. 11,1 % (poids `v005`$/10^6$, fichier `KR` ; même remarque que le groupe 6 sur le rapport officiel).
Non supervisé : typologie des couples mère–enfant. Mêmes variables actives que le groupe 6. On insiste ici sur la typologie : choix du nombre de classes sur le dendrogramme, consolidation par k-means, description et nom de chaque classe, taux pondéré d'insuffisance pondérale par classe.

```
# Groupe 7 - Insuffisance pondérale (fichier KR, un enfant par ligne)
enfant = ["caseid", "midx", "v005", "b0", "b4", "b5", "bord", "b11", "m18",
          "h11", "hw1", "hw71"]
mere = ["v012", "v106", "v190", "v025", "v024", "v136", "v113", "v116",
        "v438", "v445"]
kr = lire_dhs("KR", enfant + mere)

# Population : enfants vivants mesurés ; hw71 >= 9996 = hors limites ou manquant
df = kr[(kr["b5"] == 1) & (kr["hw71"] < 9996)].copy()
df["poids"] = df["v005"] / 1e6

# Cible : score poids-pour-âge (x 100) inférieur à -2 écarts types
df["insuff_pond"] = (df["hw71"] < -200).astype(int)
print(len(df), "enfants mesurés ; insuffisance pondérale :",
      round(taux_pondere(df["insuff_pond"], df["poids"]), 1), "%")

# Codes spéciaux et recodages
df["taille_mere"] = manquant(df["v438"], range(9990, 10000)) / 10    # en cm
df["imc_mere"] = manquant(df["v445"], range(9990, 10000)) / 100      # en kg/m2
df["m18"] = manquant(df["m18"], [8, 9])   # taille à la naissance : 1 très gros ... 5
df["premier_ne"] = df["b11"].isna().astype(int)  # pas d'intervalle pour un premier-né
df["jumeau"] = (df["b0"] > 0).astype(int)
df["diarrhee"] = indicatrice(df["h11"], [1, 2], manquants=(8, 9))
df["eau_amelioree"] = indicatrice(df["v113"], [11, 12, 13, 14, 21, 31, 41, 51, 71])
df["wc_ameliore"] = indicatrice(df["v116"], [11, 12, 13, 15, 21, 22, 41])
df["urbain"] = (df["v025"] == 1).astype(int)

# Base du supervisé ; à écarter : hw71, hw70, hw72 (indices), hw2, hw3 (mesures)
X, y, w = base_supervisee(df, "insuff_pond",
    numeriques=["hw1", "bord", "b11", "m18", "v012", "v136", "taille_mere", "imc_mere",
                "premier_ne", "jumeau", "diarrhee", "eau_amelioree", "wc_ameliore"],
    categorielles=["b4", "v106", "v190", "v025", "v024"])
resume(X, y, w)

# Base du non supervisé : enfant, mère, ménage ; la cible en illustratif
actives = ["hw1", "bord", "m18", "taille_mere", "imc_mere", "v106", "v190", "urbain",
           "v136", "eau_amelioree", "wc_ameliore"]
Z, illus = base_non_supervisee(df, actives, ["insuff_pond", "poids"])
display(X.head(), Z.describe().T.round(2))
```

Groupe 8 – Contraception moderne

Question. Quelles femmes en union utilisent une méthode moderne de contraception ? Le but est de comprendre les freins à l'utilisation.
Population. Femmes du fichier `IR` actuellement en union (`v502` $= 1$: mariées ou vivant avec un partenaire) : 6 750 femmes. Poids : `v005`$/10^6$.
Cible. `v313` donne le type de méthode utilisée actuellement : 0 aucune, 1 méthode folklorique, 2 traditionnelle (retrait, continence périodique), 3 moderne (pilule, DIU, injection, implant, préservatif, stérilisation…). `moderne` $= 1$ si `v313` $= 3$, 0 sinon.
Explicatives suggérées.

- femme : âge `v012`, instruction `v106`, religion `v130`, travail actuel `v714` (0/1), âge à la première union `v511`, exposition aux médias `v157` à `v159`, décision sur sa propre santé (`v743a` $\in \{1, 2, 3\}$: seule ou avec son conjoint ou une autre personne) ;
- fécondité : enfants nés vivants `v201`, enfants vivants `v218` ;
- couple : type d'union `v501`, instruction du conjoint `v701`, polygamie (`v505` > 0 autres épouses ; 98 : ne sait pas) ;
- ménage : quintile `v190`, taille `v136`, milieu `v025`, département `v024`.

À écarter. La méthode utilisée `v312`, l'usage passé et les intentions (`v361` à `v364`), les besoins non satisfaits `v626a`, et le désir d'enfants `v602`, `v605`, qui ont une modalité « stérilisée » : elle annonce une méthode moderne.
Taux de contrôle. 20,0 % des femmes en union (poids `v005`$/10^6$).
Non supervisé : typologie des femmes en union. Variables actives : âge, instruction, quintile, milieu urbain, enfants nés vivants et vivants, âge à la première union, médias, travail, instruction du conjoint, polygamie, décision sur sa santé. Questions : quels profils de femmes se dégagent (jeunes urbaines instruites, femmes rurales à forte fécondité, unions polygames…) ? Quel est le taux de contraception moderne de chaque classe ?

```python
# Groupe 8 - Contraception moderne (fichier IR, une femme de 15 à 49 ans par ligne)
femme = ["caseid", "v005", "v502", "v501", "v313", "v012", "v106", "v190", "v025",
         "v024", "v130", "v136", "v201", "v218", "v157", "v158", "v159", "v714",
         "v511", "v743a", "v701", "v505"]
ir = lire_dhs("IR", femme)

# Population : femmes actuellement en union (v502 = 1 : mariées ou en concubinage)
df = ir[ir["v502"] == 1].copy()
df["poids"] = df["v005"] / 1e6                    # poids de sondage de la femme

# Cible : méthode moderne utilisée actuellement (v313 = 3)
df["moderne"] = (df["v313"] == 3).astype(int)
print(len(df), "femmes en union ; contraception moderne :",
      round(taux_pondere(df["moderne"], df["poids"]), 1), "%")

# Codes spéciaux et recodages
for v in ["v157", "v158", "v159"]:
    df[v] = manquant(df[v], [9])
df["v714"] = manquant(df["v714"], [9])            # travaille actuellement : 0 / 1
df["v701"] = manquant(df["v701"], [8, 9])         # instruction du conjoint : 0 à 3
df["v130"] = manquant(df["v130"], [99])
df["polygamie"] = indicatrice(df["v505"], range(1, 98), manquants=(98, 99))
df["decide_sante"] = indicatrice(df["v743a"], [1, 2, 3], manquants=(9,))
df["urbain"] = (df["v025"] == 1).astype(int)

# Base du supervisé ; à écarter : v312, v361 à v364 (méthode, usage passé, prévu),
# v626a (besoins non satisfaits), v602 et v605 (contiennent « stérilisée »)
X, y, w = base_supervisee(df, "moderne",
    numeriques=["v012", "v136", "v201", "v218", "v511", "v157", "v158", "v159",
                "v714", "polygamie", "decide_sante"],
    categorielles=["v106", "v190", "v025", "v024", "v130", "v501", "v701"])
resume(X, y, w)
```

```python
# Base du non supervisé : profil des femmes ; la cible en illustratif
actives = ["v012", "v106", "v190", "urbain", "v201", "v218", "v511", "v157", "v158",
           "v159", "v714", "v701", "polygamie", "decide_sante"]
Z, illus = base_non_supervisee(df, actives, ["moderne", "poids"])
display(X.head(), Z.describe().T.round(2))
```

Groupe 9 – Besoins non satisfaits en contraception

Question. Quelles femmes en union voudraient éviter ou espacer une grossesse sans utiliser de contraception ? Ce sont les femmes que les programmes de planification familiale cherchent à atteindre.
Population. Femmes du fichier `IR` actuellement en union (`v502` $= 1$) : 6 750 femmes. Poids : `v005`$/10^6$.
Cible. `v626a` est l'indicateur calculé par DHS (définition révisée) : 1 besoin non satisfait pour espacer, 2 pour limiter, 3 et 4 utilise une méthode pour espacer ou limiter, 7 pas de besoin, 9 stérile ou ménopausée (codes à vérifier dans le `.MAP`). `besoin_ns` $= 1$ si `v626a` $\in \{1, 2\}$, 0 pour toutes les autres femmes en union, y compris les stériles.
Explicatives suggérées. Celles du groupe 8.
À écarter. Tout ce qui sert à construire l'indicateur : méthode et type de méthode (`v312`, `v313`), usage et intentions (`v361` à `v364`), désir d'enfants (`v602`, `v605`), grossesse actuelle et désir de cette grossesse (`v213`, `v225`), aménorrhée et abstinence post-partum (`v405`, `v406`).
Taux de contrôle. 18,4 % des femmes en union (poids `v005`$/10^6$).
Non supervisé : où sont les besoins non satisfaits ? Mêmes variables actives que le groupe 8. On insiste sur l'ACP : nommer les deux premiers axes, projeter les femmes colorées selon la cible, puis typologie et taux de besoins non satisfaits par classe. Comparer avec le groupe 8 n'est pas demandé, mais la discussion peut citer les profils où la contraception est faible *et* le besoin élevé.

```python
# Groupe 9 - Besoins non satisfaits en contraception (fichier IR, une femme par ligne)
femme = ["caseid", "v005", "v502", "v501", "v626a", "v012", "v106", "v190", "v025",
         "v024", "v130", "v136", "v201", "v218", "v157", "v158", "v159", "v714",
         "v511", "v743a", "v701", "v505"]
ir = lire_dhs("IR", femme)

# Population : femmes actuellement en union (v502 = 1 : mariées ou en concubinage)
df = ir[ir["v502"] == 1].copy()
df["poids"] = df["v005"] / 1e6                    # poids de sondage de la femme

# Cible : besoin non satisfait pour espacer (v626a = 1) ou pour limiter (v626a = 2)
df["besoin_ns"] = df["v626a"].isin([1, 2]).astype(int)
print(len(df), "femmes en union ; besoins non satisfaits :",
      round(taux_pondere(df["besoin_ns"], df["poids"]), 1), "%")

# Codes spéciaux et recodages
for v in ["v157", "v158", "v159"]:
    df[v] = manquant(df[v], [9])
df["v714"] = manquant(df["v714"], [9])            # travaille actuellement : 0 / 1
df["v701"] = manquant(df["v701"], [8, 9])         # instruction du conjoint : 0 à 3
df["v130"] = manquant(df["v130"], [99])
df["polygamie"] = indicatrice(df["v505"], range(1, 98), manquants=(98, 99))
df["decide_sante"] = indicatrice(df["v743a"], [1, 2, 3], manquants=(9,))
df["urbain"] = (df["v025"] == 1).astype(int)

# Base du supervisé ; à écarter : ce qui construit v626a (v312, v313, v361 à v364,
# v602, v605, grossesse v213, v225, aménorrhée v405, abstinence v406)
X, y, w = base_supervisee(df, "besoin_ns",
    numeriques=["v012", "v136", "v201", "v218", "v511", "v157", "v158", "v159",
                "v714", "polygamie", "decide_sante"],
    categorielles=["v106", "v190", "v025", "v024", "v130", "v501", "v701"])
resume(X, y, w)

# Base du non supervisé : profil des femmes ; la cible en illustratif
actives = ["v012", "v106", "v190", "urbain", "v201", "v218", "v511", "v157", "v158",
           "v159", "v714", "v701", "polygamie", "decide_sante"]
Z, illus = base_non_supervisee(df, actives, ["besoin_ns", "poids"])
```

```
display(X.head(), Z.describe().T.round(2))
```

5 Accès aux données pour le cours : l'autorisation de classe

À retenir

Pour ce cours, l'enseignant demande au programme DHS une **autorisation pour toute la classe**. Les étudiants n'ont pas de compte à créer : les fichiers leur sont remis par l'enseignant, pour le seul projet du cours, dans les conditions fixées par l'accord écrit de DHS. Ce qui est rendu, c'est le code et le rapport, jamais les données.

La procédure suivie par l'enseignant

Elle est décrite ici pour qu'un futur enseignant, ou un étudiant devenu enseignant, puisse la reprendre.

1. **Créer son compte** d'utilisateur DHS (voir la section 6 pour les précautions d'inscription).
2. **Repérer l'option « Instructors »** : sur la page de création d'un projet, DHS indique que les enseignants peuvent faire autoriser leur classe sans inscription individuelle des étudiants, sur demande par courriel.
3. **Écrire à** `archive@dhsprogram.com` depuis l'adresse du compte, en précisant : l'établissement et le cours ; l'effectif (avec la liste nominative des étudiants si DHS la demande) ; l'enquête et les fichiers (Congo DHS 2011-12, fichiers ménages `HR`, femmes `IR`, enfants `KR`, format Stata ou SPSS) ; l'usage (projets de groupe encadrés) ; les dates (octobre à décembre 2026) ; les règles imposées aux étudiants.
4. **Attendre la réponse** de DHS, qui fixe les conditions de l'autorisation, puis télécharger les fichiers depuis le compte de l'enseignant.
5. **Remettre les fichiers** aux seuls étudiants de la liste transmise à DHS, par un canal privé : courriel en copie cachée aux seules adresses autorisées, clé USB en salle, ou dossier partagé restreint à la classe. Jamais de dépôt public. Chaque étudiant reçoit les conditions d'utilisation de DHS (`https://dhsprogram.com/data/terms-of-use.cfm`) et signe la charte ci-dessous, de préférence avant la remise des fichiers.
6. **Déclarer tout nouvel étudiant** à DHS (nom et adresse électronique) *avant* de lui remettre les fichiers.
7. **Clore** : à la fin du cours, rappeler aux étudiants de supprimer leurs copies, sauf autre disposition de l'accord DHS.

Charte d'utilisation des données DHS – à signer par chaque étudiant

Je soussigné(e) ..... , étudiant(e) du cours *Introduction au Machine Learning* (CNFSDP, 2026-2027), reçois de l'enseignant les fichiers de l'Enquête démographique et de santé du Congo 2011-2012, mis à disposition par le programme DHS au titre d'une autorisation de classe. Je m'engage à :

— n'utiliser ces données que pour le projet de groupe du cours ;

— ne les transmettre à personne, pas même à un autre étudiant ou à un autre groupe :

chacun reçoit sa copie de l'enseignant ;

— ne jamais les déposer sur Internet (GitHub, messagerie, réseaux sociaux, dossier partagé public) ;

— ne tenter d'identifier aucune personne ni aucun ménage ;

— supprimer mes copies à la fin du cours.

Date : _____ Signature : _____

6 Pour plus tard : faire sa propre demande DHS

Après le cours, pour un mémoire, un stage ou une recherche, chacun peut obtenir les données DHS *en son nom*. L'autorisation de classe ne couvre que le projet du cours.

1. **Créer son compte** : https://dhsprogram.com/data/new-user-registration.cfm
   — sur un **ordinateur** (le formulaire ne fonctionne pas sur téléphone) ;
   — **sans traduction automatique** : si le navigateur propose de traduire la page, refuser (« Afficher l'original ») et recharger la page ;
   — **tous les champs en anglais et sans accents**, par exemple *Centre National de Formation en Statistique, Demographie et Planification*, écrit en entier ;
   — pays de résidence : *Congo* (et non *Congo Democratic Republic*) ; téléphone avec l'indicatif `+242` ; mot de passe de 8 à 15 caractères ;
   — une adresse électronique que vous consultez : elle sert d'identifiant, et l'accord arrive par courriel.
2. **Créer un projet** (*Create New Project*) : titre, co-chercheurs éventuels (facultatif), description de l'analyse prévue (modèle ci-dessous, à adapter).
3. **Choisir le pays et l'enquête**, puis les jeux de données d'enquête (*survey datasets*) ; les données GPS ou biomarqueurs demandent des engagements supplémentaires.
4. **Attendre l'accord**, en général sous 24 à 48 heures ouvrées, puis télécharger. Instructions officielles : https://dhsprogram.com/data/Access-Instructions.cfm

Si la connexion échoue sans message d'erreur. Repartir de la page https://dhsprogram.com/data/dataset_admin/login_main.cfm sans rien ajouter à l'adresse ; effacer les cookies du site `dhsprogram.com` ou ouvrir une fenêtre de navigation privée ; désactiver la traduction ; vérifier les majuscules du mot de passe ; à défaut, utiliser « mot de passe oublié » ou écrire à `archive@dhsprogram.com`.

Modèle de description de projet (en anglais, sans accents ; remplacer les crochets)

Project Title :
[Short title, for example : Determinants of child vaccination in the Republic of the Congo]

Description :

I am [a master student / a statistician] at [full name of the institution], [city, country]. I will use the [survey, for example Congo DHS 2011-12] [files, for example household, women and children recode files] to study [research question]. Methods : [for example logistic regression, random forests, principal component analysis]. The results will be presented in [a thesis / a report / an article]. The data will be kept on my own computer and will not be shared or published.